



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And
Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Title: Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS)

Objective

The objective of this lab is to understand Cross-Site Scripting (XSS) vulnerabilities, demonstrate how XSS attacks occur, and learn how to prevent them using proper input handling techniques.

Introduction to Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a web security vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users. These scripts execute in the victim's browser and can be used to steal session cookies, hijack user accounts, redirect users to malicious websites, or perform actions on behalf of the user.

XSS commonly occurs when user input is accepted and displayed by a web application without proper validation or output encoding.

Exploiting the XSS Vulnerability

Description of the Vulnerability

In this lab, a simple PHP web application was created with an input form that accepts a user's name and displays it back to the user. The application directly outputs user input without sanitization, making it vulnerable to Cross-Site Scripting attacks.

Exploitation Process

1. A form was created in index.php to accept user input.
2. The input was processed and displayed in xss_form.php without validation or encoding.

A malicious JavaScript payload was entered into the input field:

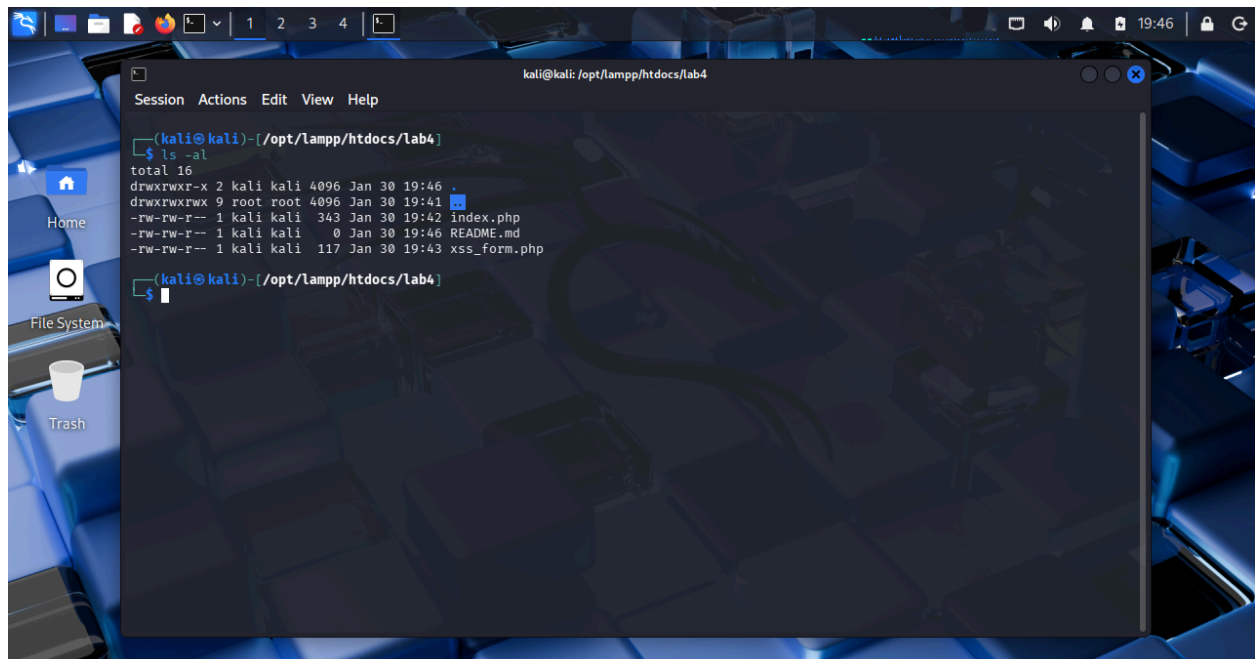
```
<script>alert('XSS Attack!');</script>
```

- 3.
4. When the form was submitted, the script executed in the browser and displayed an alert box.

Result

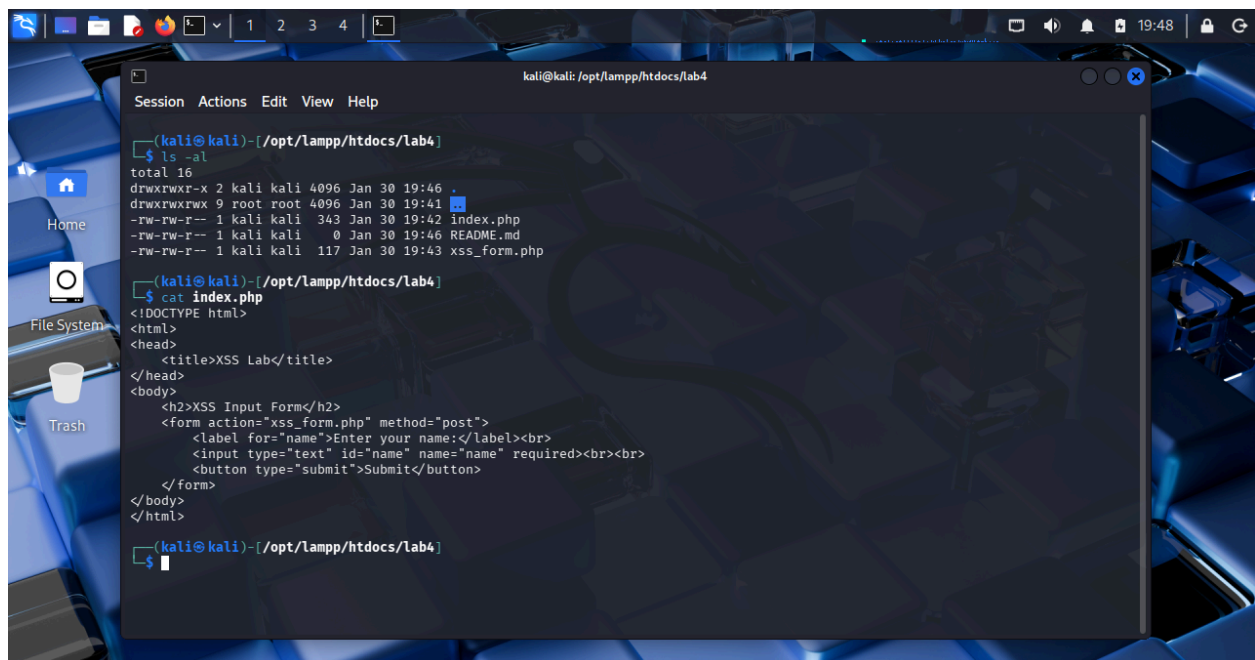
The successful execution of the JavaScript alert confirmed the presence of an XSS vulnerability. This demonstrated how attackers can inject and execute malicious scripts through unsanitized user input.

Below is the lab folder and files within the folder, exploitations



A terminal window titled 'kali@kali: /opt/lampp/htdocs/lab4' displays the output of the 'ls -al' command. The output shows a directory listing with permissions, owner, group, size, date, and filename. The files listed are 'index.php', 'README.md', and 'xss_form.php'.

```
kali@kali: /opt/lampp/htdocs/lab4
$ ls -al
total 16
drwxrwxr-x 2 kali kali 4096 Jan 30 19:46 .
drwxrwxrwx 9 root root 4096 Jan 30 19:41 ..
-rw-rw-r-- 1 kali kali 343 Jan 30 19:42 index.php
-rw-rw-r-- 1 kali kali 0 Jan 30 19:46 README.md
-rw-rw-r-- 1 kali kali 117 Jan 30 19:43 xss_form.php
```



A terminal window titled 'kali@kali: /opt/lampp/htdocs/lab4' displays the output of the 'cat index.php' command. The output shows the HTML structure of the file, including a title 'XSS Lab', a form with a text input field, and a submit button.

```
kali@kali: /opt/lampp/htdocs/lab4
$ cat index.php
<!DOCTYPE html>
<html>
<head>
<title>XSS Lab</title>
</head>
<body>
<h2>XSS Input Form</h2>
<form action="xss_form.php" method="post">
<label for="name">Enter your name:</label><br>
<input type="text" id="name" name="name" required><br><br>
<button type="submit">Submit</button>
</form>
</body>
</html>
```

```
kali@kali: /opt/lampp/htdocs/lab4

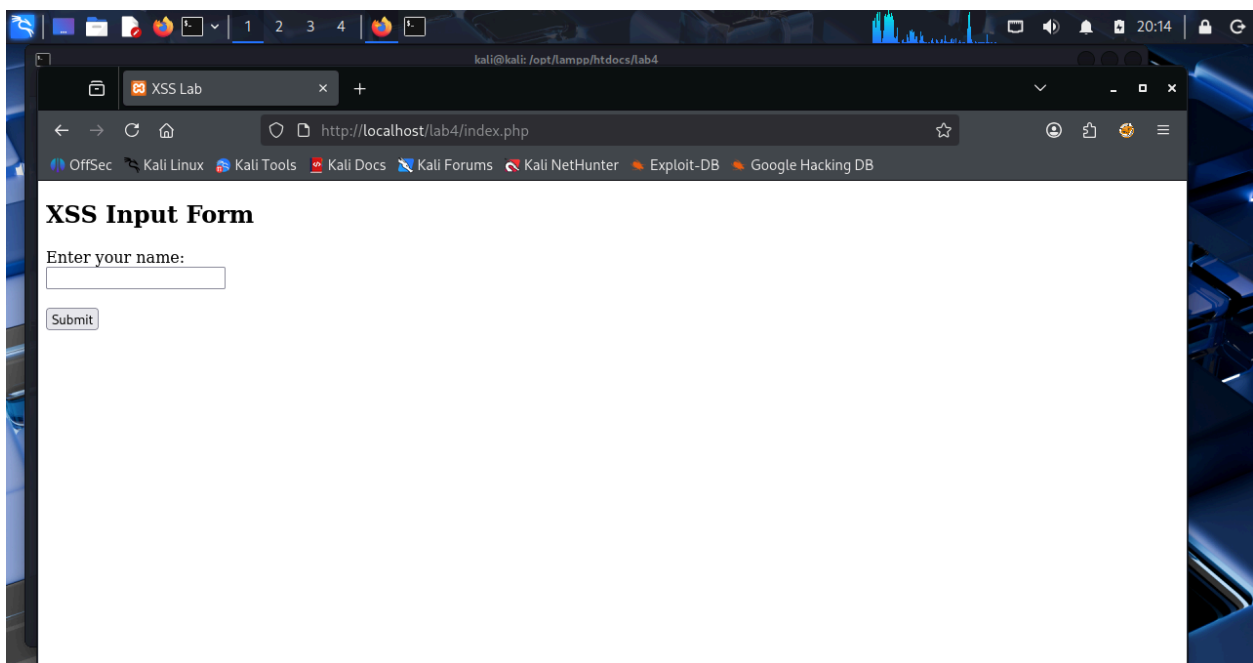
Session Actions Edit View Help
-rw-rw-r-- 1 kali kali 0 Jan 30 19:46 README.md
-rw-rw-r-- 1 kali kali 117 Jan 30 19:43 xss_form.php

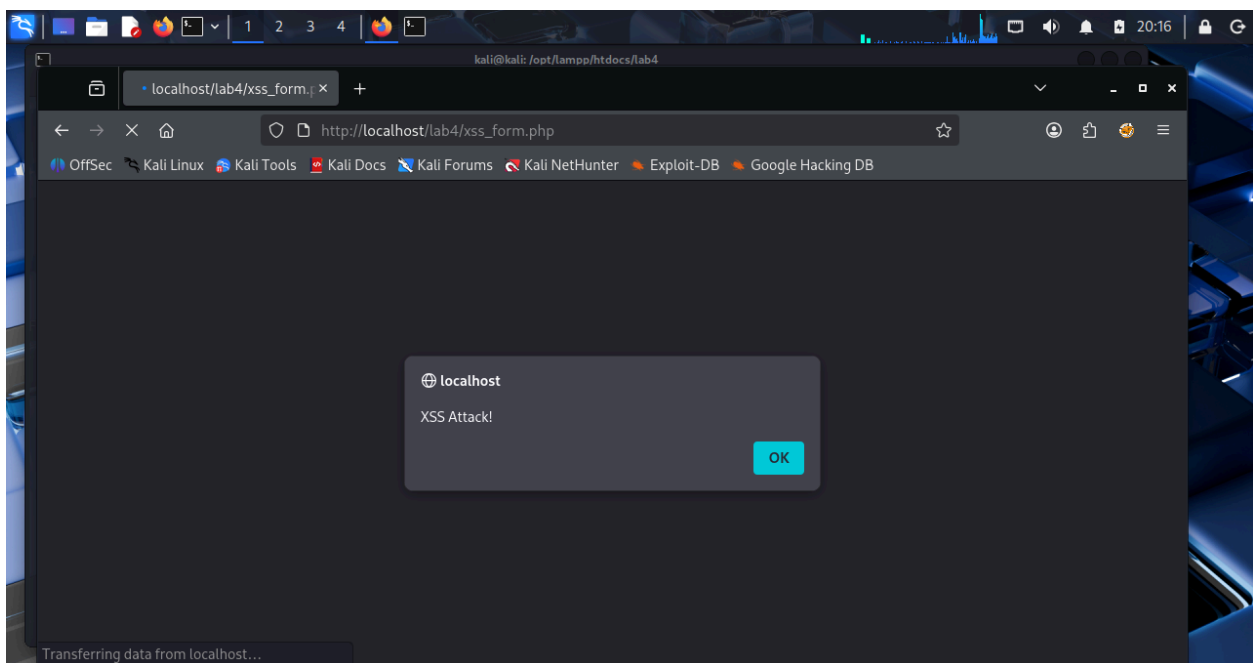
(kali@kali)-[/opt/lampp/htdocs/lab4]
$ cat index.php
<!DOCTYPE html>
<html>
<head>
<title>XSS Lab</title>
</head>
<body>
<h2>XSS Input Form</h2>
<form action="xss_form.php" method="post">
<label for="name">Enter your name:</label><br>
<input type="text" id="name" name="name" required><br><br>
<button type="submit">Submit</button>
</form>
</body>
</html>

(kali@kali)-[/opt/lampp/htdocs/lab4]
$ cat xss_form.php
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
$name = $_POST['name'];

echo "<p>Hello, $name!</p>";
}
?>

(kali@kali)-[/opt/lampp/htdocs/lab4]
$
```





Mitigating the XSS Vulnerability

Mitigation Technique Used

To prevent the XSS attack, output encoding was applied using the PHP `htmlspecialchars()` function. This function converts special characters such as `<`, `>`, and quotes into their HTML entity equivalents, preventing the browser from interpreting them as executable code.

Implementation

The vulnerable code in `xss_form.php` was modified to sanitize user input before displaying it:

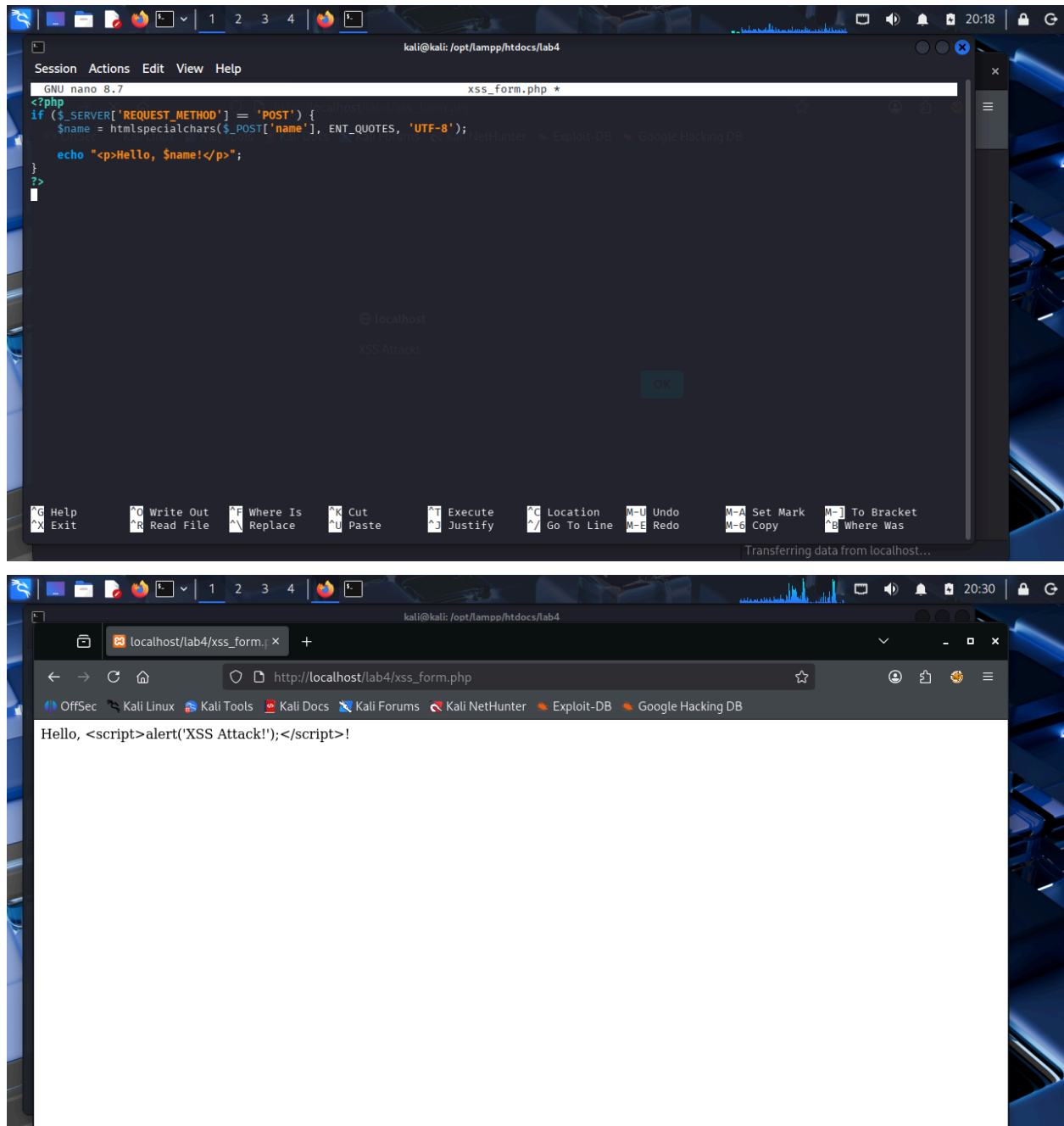
- User input was passed through `htmlspecialchars()` with proper encoding settings.
- The sanitized input was then safely displayed on the page.

Testing the Mitigation

After applying the fix, the same malicious payload was submitted again. This time:

- The script did not execute.
- The payload was displayed as plain text.
- No JavaScript alert appeared.

This confirmed that the XSS vulnerability was successfully mitigated.



Importance of Input Validation and Output Encoding

Input validation and output encoding are critical security practices in web development. They ensure that untrusted user input does not get interpreted as executable code by the browser. Proper use of these techniques helps protect users from client-side attacks and prevents common vulnerabilities such as Cross-Site Scripting.

Failing to sanitize user input can expose applications to serious security risks, including data theft and session hijacking.

Conclusion

This lab demonstrated how Cross-Site Scripting vulnerabilities occur and how easily they can be exploited when user input is not handled securely. It also showed that XSS can be effectively prevented by applying proper output encoding techniques such as `htmlspecialchars()`. Developers must always treat user input as untrusted and apply appropriate security controls to protect web applications.